Transport Layer Specification

Hardware-Agnostic ISO/IEC 14443-4 (ISO-DEP) Bridge
for the NFC Smart Lock Protocol

Version 0.3 – Draft

July 24, 2026

Abstract

This document specifies the transport layer that carries the cryptographic handshake and secure-messaging protocol defined in *smartlock_session_layer.pdf* over any ISO/IEC 14443-4 (ISO-DEP) compliant contactless link. It is intentionally **hardware-agnostic**: it defines APDU framing, the instruction set, the target-side protocol state machine, timing requirements, and status codes purely in terms of the ISO/IEC 14443-4 standard and ISO/IEC 7816-4 APDU conventions. It does *not* reference any specific NFC controller chip, command set, or register. Binding this specification to a concrete controller (e.g. PN532, ST25R3911B, CR95HF) is the responsibility of a separate *Hardware/Link Layer* document, which implements the abstract interface defined in Section 2 for that controller.

Contents

1 Scope and Layering

1.1 Purpose

This specification is deliberately independent of any single NFC front-end IC. The Lock's application logic (handshake state machine, APDU parsing, status word generation) should compile and run unchanged against any controller that can act as an ISO/IEC 14443-4 Type A PICC (Target) at 106 kbps, provided that controller's driver implements the **Link Layer Interface (LLI)** defined in Section 2.

1.2 Layer Responsibilities

Layer	Responsibility
Session Layer	Ed25519 / X25519 / HKDF / AES-GCM handshake and secure messaging. Defined in *smartlock_session_layer.pdf*. Operates on opaque byte payloads; has no knowledge of NFC, APDUs, or RF.
Application Layer	Command dispatch (`CMD_UNLOCK`), authorization policy, lock actuation logic. Defined in *smartlock_application_layer.pdf*. Receives plaintext from the session layer; has no knowledge of cryptography or transport.
Transport Layer (this document)	APDU framing, instruction set, protocol-level state machine, ISO-DEP timing (FWT/WTX budget), status word dictionary, chaining policy. Depends only on ISO/IEC 14443-4 and ISO/IEC 7816-4 semantics via the LLI.
Hardware / Link Layer	Concrete controller driver (command opcodes, register access, bus protocol – SPI/I2C/HSU, firmware-specific status codes, RF activation sequencing). Implements the LLI for one specific chip. Out of scope for this document.

A change of NFC controller should require changes *only* to the Hardware/Link Layer document and its corresponding driver – never to this document or to the cryptographic protocol.

2 Link Layer Interface (LLI)

The transport layer depends on a minimal, controller-independent interface. Any conforming ISO-DEP controller driver MUST implement these five primitives. Exact function signatures are a Hardware/Link Layer concern; only behavior is normative here.

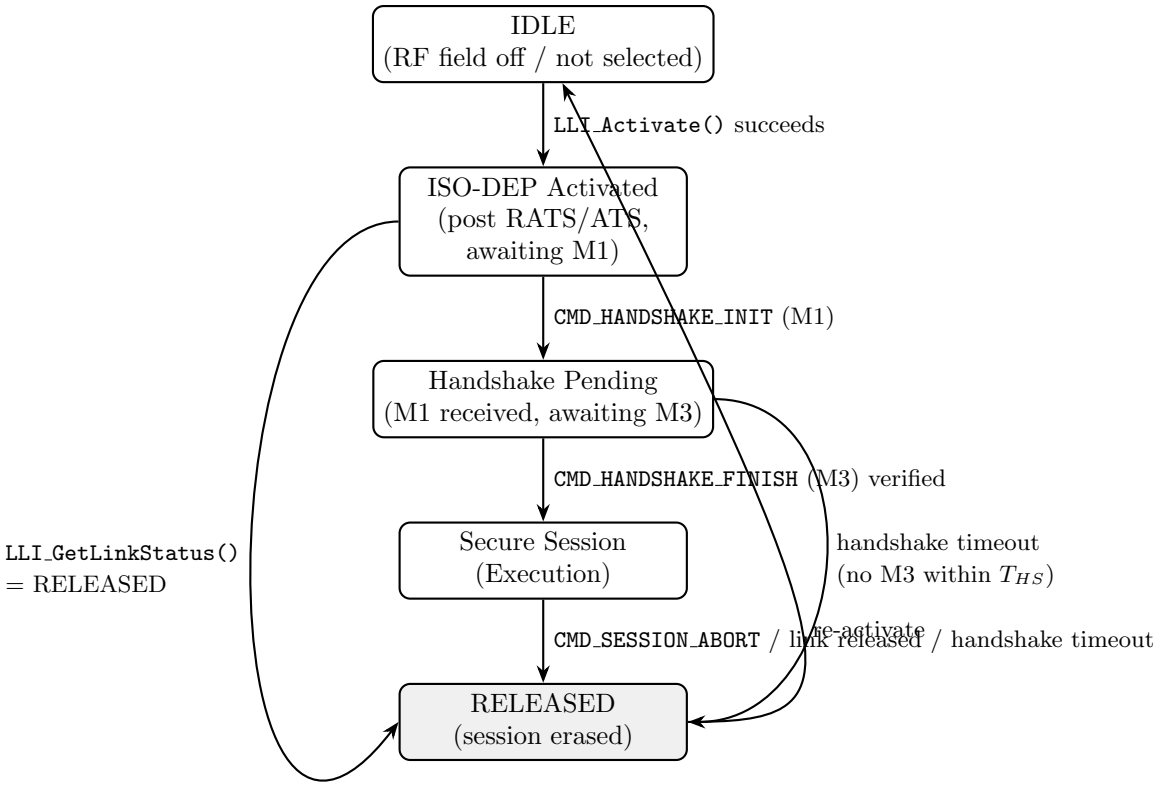
Primitive	Direction	Required Behavior
`LLI_Activate()`	Lock-internal	Performs ISO/IEC 14443-3 polling/anti-collision/selection and ISO/IEC 14443-4 RATS/ATS activation. Returns once the link is in the *ISO-DEP Activated* protocol state (Section 3) or reports failure.

Primitive	Direction	Required Behavior
`LLI_ReceiveAPDU(timeout)`	Target ← Initiator	Blocks (up to `timeout`) for the next C-APDU. Returns raw APDU bytes, or a defined error (timeout, frame-integrity error, link released).
`LLI_SendAPDU(bytes)`	Target → Initiator	Transmits an R-APDU. If the ISO-DEP frame-waiting-time budget is at risk of expiry due to caller processing delay, the driver is responsible for issuing ISO/IEC 14443-4 `S(WTX)` waiting-time-extension blocks per the standard; this is transparent to the transport layer.
`LLI_GetLinkStatus()`	Lock-internal	Returns one of: `ACTIVE`, `RELEASED`, `ERROR`. MUST reflect RF field loss or explicit deselection promptly enough to satisfy the polling requirement in Section 8.3.
`LLI_Abort()`	Lock-internal	Forces immediate link teardown (deselect / halt) regardless of current protocol state.

Non-normative note: the abstract error taxonomy returned by `LLI_ReceiveAPDU` (timeout / frame-integrity error / link released) is intentionally coarse. A Hardware/Link Layer document MUST map its controller's native error/status codes onto this taxonomy; it MUST NOT require the transport layer to understand controller-specific codes.

3 Target-Side Protocol State Machine

States are named in terms of ISO/IEC 14443-3/4 protocol phases plus the application states introduced by this specification. No state below implies any particular controller command.



The *Handshake Pending* intermediate state is explicit: a session entering this state that does not receive a verified M3 within the handshake timeout T_{HS} (Section 8.1) transitions directly to *RELEASED* with all ephemeral cryptographic material erased, identically to a link-loss event.

4 Data Framing and APDU Encapsulation

Framing follows ISO/IEC 7816-4 *Short APDU* (Case 4) conventions, as used generically over any ISO/IEC 14443-4 link.

4.1 Command APDU (C-APDU): Phone → Lock

Field	Size	Value / Description
CLA	1 byte	`0x80` (proprietary class)
INS	1 byte	`0x10` / `0x11` / `0x20` / `0x30` (Section 5)
P1	1 byte	`0x00` (reserved)
P2	1 byte	`0x00` (reserved)
Lc	1 byte	Length of data payload, `0x01`–`0xFF`
Data	*Lc* bytes	Handshake parameters or AES-GCM secure payload

A Short APDU C-APDU (Case 4) is bounded at **261 bytes** (CLA+INS+P1+P2+Lc+255 data bytes+Le) per ISO/IEC 7816-4. This bound applies to any controller supporting Short APDU exchange and is not specific to this deployment's hardware.

4.2 Response APDU (R-APDU): Lock → Phone

Field	Size	Value / Description
Data	0–256 bytes	Handshake response or encrypted application response
SW1	1 byte	`0x90` = normal processing; see Section 9
SW2	1 byte	Status sub-code

An R-APDU is bounded at **258 bytes** (256 data bytes + SW1 + SW2), per the same Short APDU convention.

4.3 Maximum Transmission Unit and Chaining Budget

Handshake payloads fit within a single frame under any ISO-DEP controller:

Message	Payload Size	Fits in Single Frame?
M1 ($pk_P^{eph} \| c_P$)	64 bytes	Yes
M2 ($pk_L^{eph} \| c_L \| \mathrm{Sig}_L$)	128 bytes	Yes
M3 (Sig_P)	64 bytes	Yes

For `CMD_SECURE_PAYLOAD`, the usable plaintext budget per *unchained* message:

```
Max Lc (Short APDU data field)          = 255 bytes
Nonce overhead                          = 12 bytes
GCM Auth Tag overhead                   = 16 bytes
-------------------------------------------------------
Maximum plaintext per unchained message = 255 - 12 - 16 = 227 bytes
```

Application-layer chaining is explicitly out of scope for this revision. A payload requiring more than 227 bytes of plaintext MUST be rejected with status word `0x6A 0x80`, not silently truncated or chained. ISO/IEC 14443-4 native I-block chaining, if exposed by a given controller's driver, MAY be used transparently underneath a single `LLI_SendAPDU`/`LLI_ReceiveAPDU` call in a future revision, but that mechanism is a Hardware/Link Layer concern, not a transport-layer one.

5 Instruction Set

Instruction	INS	Description
`CMD_HANDSHAKE_INIT`	`0x10`	Carries M1 ($pk_P^{eph} \| c_P$), 64 bytes
`CMD_HANDSHAKE_FINISH`	`0x11`	Carries M3 (Sig_P), 64 bytes
`CMD_SECURE_PAYLOAD`	`0x20`	Carries AES-256-GCM encrypted application command (nonce ‖ ciphertext ‖ tag)
`CMD_SESSION_ABORT`	`0x30`	Explicit session teardown; triggers immediate key erasure regardless of current state

CLA is `0x80` and P1/P2 are `0x00` for all four instructions.

5.1 State / Instruction Validity

Instruction	Valid State	If Invalid	Effect
`CMD_HANDSHAKE_INIT`	ISO-DEP Activated (awaiting M1) only	`0x69 0x85`	None (no state change)
`CMD_HANDSHAKE_FINISH`	Handshake Pending only	`0x69 0x85`	None if received early; `0x69 0x82` + erase if signature invalid
`CMD_SECURE_PAYLOAD`	Secure Session only	`0x69 0x85`	None (no state change)
`CMD_SESSION_ABORT`	Any state from ISO-DEP Activated onward	n/a (always accepted)	Immediate erase, transition to `RELEASED`

A repeated `CMD_HANDSHAKE_INIT` while already in *Handshake Pending* or *Secure Session* MUST be treated as invalid (`0x69 0x85`); restarting a session requires the link to be re-activated from *IDLE* via `LLI_Activate()`.

6 Phase 1: Handshake and Key Agreement

6.1 Exchange 1: M1 → M2

Phone → Lock (`CMD_HANDSHAKE_INIT`): CLA `0x80`, INS `0x10`, Lc `0x40` (64 bytes). Data: 32 bytes pk_P^{eph} ‖ 32 bytes c_P.

Lock → Phone (R-APDU, M2): 128 bytes data (32 bytes pk_L^{eph} ‖ 32 bytes c_L ‖ 64 bytes Sig_L), SW1/SW2 = `0x90 0x00`.

Receipt of M1 transitions the Lock to *Handshake Pending* and starts the handshake timer T_{HS} (Section 8.1).

6.2 Exchange 2: M3 → Authentication Status

Phone → Lock (`CMD_HANDSHAKE_FINISH`): CLA `0x80`, INS `0x11`, Lc `0x40` (64 bytes). Data: 64 bytes Sig_P.

Lock → Phone (R-APDU): empty data field. SW1/SW2 = `0x90 0x00` on successful verification (transition to *Secure Session*, HKDF derivation on both sides), or `0x69 0x82` on failure (transition to *RELEASED* with immediate erase).

7 Phase 2: Secure Communication

Once in *Secure Session*, all application payloads use `CMD_SECURE_PAYLOAD` (`0x20`).

Phone → Lock: Lc = variable, up to `0xFF`. Data: 12-byte nonce ‖ ciphertext (encrypted with K_{p2e}) ‖ 16-byte auth tag.

Lock → Phone: Data: 12-byte nonce ‖ ciphertext (encrypted with K_{e2p}) ‖ 16-byte auth tag. SW1/SW2 = `0x90 0x00` on success.

An AES-GCM authentication tag mismatch MUST be treated identically to a handshake signature failure: respond `0x69 0x82` and invoke the secure-erase routine immediately.

8 Timing, Liveness, and Session Guardrails

8.1 Handshake Timeout

The Lock MUST enforce a maximum duration T_{HS} between entering *Handshake Pending* (receipt of M1) and receiving a verified M3. A recommended starting value is $T_{HS} = 2\,\text{s}$, comfortably above software Ed25519 signing/verification latency (single-digit milliseconds) and typical NFC round-trip overhead. This value is a transport-layer policy choice and does not depend on the underlying controller; a Hardware/Link Layer document MAY document how its controller's own frame-waiting-time (FWT) and `S(WTX)` behavior interacts with T_{HS}, but MUST NOT change T_{HS} itself.

8.2 Waiting Time Extension (Protocol-Level)

ISO/IEC 14443-4 defines an `S(WTX)` block allowing a Target to request additional processing time from the Initiator when a response cannot be produced within the negotiated Frame Waiting Time (FWT), derived from the FWI value exchanged during ATS. This specification requires only that:

- WTX handling, if needed, is issued transparently by the Hardware/Link Layer implementation of `LLI_SendAPDU` – the transport layer neither issues nor observes `S(WTX)` blocks directly.
- Application processing between `LLI_ReceiveAPDU` and `LLI_SendAPDU` SHOULD remain well under one second wherever possible, since WTX extension is bounded in practice by Initiator-side retry limits regardless of controller. Ed25519 signing on the current ESP32 target completes in low single-digit milliseconds, so this budget is not expected to be stressed by the cryptographic protocol itself.

8.3 Link Status Polling

The Lock SHOULD call `LLI_GetLinkStatus()` between application-level exchanges – at minimum immediately before committing any state-changing action (e.g. actuating the lock mechanism) – and MUST treat a `RELEASED` result identically to an explicit `CMD_SESSION_ABORT`. The maximum acceptable polling interval is a Hardware/Link Layer concern, since it depends on how quickly a given controller can detect RF loss; this document only requires that detection occur before any state-changing action is committed.

8.4 Secure-Erase Routine

The following trigger conditions all invoke the identical erase routine defined in *smartlock_session_layer.pdf* §7 (destruction of sk^{eph}, SharedSecret, PRK, and derived session keys):

- `CMD_SESSION_ABORT` received in any active state
- Handshake timeout T_{HS} expiry
- `LLI_GetLinkStatus()` returns `RELEASED`
- `LLI_ReceiveAPDU` reports a frame-integrity error during *Handshake Pending* or *Secure Session*
- Handshake signature verification failure (M2 at Phone, or M3 at Lock)
- AES-GCM authentication tag verification failure

9 Status Words (SW1/SW2) Dictionary

SW1	SW2	Description	Trigger Condition
0x90	0x00	Success	Command executed correctly
0x6A	0x80	Incorrect parameters in data field	Invalid X25519 point (e.g. all-zero), malformed Lc, or secure payload exceeding the 227-byte unchained plaintext budget
0x69	0x82	Security status not satisfied	Handshake signature verification failed, or AES-GCM auth tag mismatch – both trigger immediate secure erase
0x69	0x85	Conditions of use not satisfied	Instruction received outside its valid state per Section 5.1
0x6A	0x81	Function not supported	Unknown INS byte

These status words are application-layer (this protocol's own convention) and are transmitted as ordinary R-APDU trailer bytes; they do not depend on any controller-specific status/error reporting mechanism. A `LLI_ReceiveAPDU` frame-integrity error (Section 2) is a *link-layer* condition and is handled via the secure-erase routine directly – it never reaches the application as an SW1/SW2 pair, since no valid APDU was received to respond to.

10 Conformance Requirements for a Hardware/Link Layer Binding

A Hardware/Link Layer document for a specific controller MUST, at minimum:

1. Map its controller's activation command sequence to `LLI_Activate()`, including 106 kbps Type A Passive Card Emulation configuration equivalent to what is assumed here.
2. Map its controller's frame retrieval/transmission primitives to `LLI_ReceiveAPDU`/`LLI_SendAPDU`, including translation of controller-native error codes into the timeout / frame-integrity-error / link-released taxonomy.
3. Document how (or whether) the controller autonomously handles `S(WTX)` generation, and any bounds on that behavior (maximum single extension, maximum retries) relevant to Section 8.2.
4. Specify a concrete link-status polling mechanism and interval satisfying Section 8.3.
5. Confirm the controller supports Short APDU exchange up to the 261/258-byte bounds assumed in Section 4, or document a reduced ceiling if it does not.

11 Revision History

Version	Change	Date
0.1	Initial transport draft (APDU framing, instruction set, PN532 command sequence embedded directly); subsequently corrected malformed byte sequence and added handshake timeout, state/instruction validity table, session-abort wiring, quantified chaining budget, WTX bound, and polling guidance, still PN532-coupled at this stage	–
0.2	**Removed all controller-specific content.** Introduced the abstract Link Layer Interface (LLI); state machine, timing, and status words now expressed purely in ISO/IEC 14443-4 / ISO/IEC 7816-4 terms. PN532-specific material relocated to a separate Hardware/Link Layer document (now versioned 0.1). A concrete C implementation of the LLI is tracked separately in the LLI Engineering Specification (v0.3)	–
0.3	Updated references from combined *smartlock_protocol_spec.pdf* to split *smartlock_session_layer.pdf* and *smartlock_application_layer.pdf*. Split "Cryptographic Protocol" row in layer responsibilities table into separate Session Layer and Application Layer rows. No protocol changes.	July 24, 2026

Note on numbering: this project has not yet reached a stable v1.0 release. Version numbers below 1.0 are used throughout the document set (protocol, transport, hardware/link layer, LLI) until the first frozen release, and are not directly comparable across documents – e.g. transport layer 0.3 does not imply it is "ahead of" hardware/link layer 0.1 in maturity, only that it is this document's third drafted revision.